



FEU INSTITUTE OF TECHNOLOGY

COMPUTER SCIENCE DEPARTMENT

CS0045

CS Elective - Computer Graphics & Visual Computing

TECHNICAL ASSESSMENT

1

Module 1-4

Student Name(s):	1. 2. 3. 4.
Section:	
Professor:	

Interactive OpenGL Scene Challenge

I. OBJECTIVES

At the end of this technical assessment, students must be able to:

Cognitive: Explain and differentiate the roles of OpenGL geometric primitives, vertex arrays, GLUT callbacks, and text-rendering functions in constructing an interactive 2D graphics application.

Psychomotor: Modify a functional OpenGL starter program to create an interactive graphical scene using multiple primitives and colors, convert an object to vertex-array rendering, implement keyboard interaction, and display user instructions using GLUT text-rendering functions.

Affective: Demonstrate responsible, critical, and transparent use of AI assistance by independently validating graphical output, explaining implementation decisions, and defending or modifying the completed program when required.

II. BACKGROUND INFORMATION

In order to accomplish this assessment, the student must have a clear understanding of the following topics:

- Basic structure, initialization, and rendering flow of an OpenGL/GLUT application
- OpenGL coordinate system, vertices, and graphical object construction
- Geometric primitives such as points, lines, triangles, quadrilaterals, and polygons
- Application of colors and graphical attributes to rendered objects
- GLUT window initialization, display functions, and event-driven programming
- Keyboard callbacks using `glutKeyboardFunc()`
- Updating graphical output using `glutPostRedisplay()`
- Raster positioning and bitmap text rendering using `glRasterPos2f()` and `glutBitmapString()`
- Vertex-array representation of graphical data
- Vertex-array rendering using `glEnableClientState(GL_VERTEX_ARRAY)`, `glVertexPointer()`, and `glDrawArrays()`
- Conversion of an immediate-mode graphical object into a vertex-array implementation
- Testing and verification of graphical output and interactive behavior
- Responsible use of AI for explanation, debugging, component-level assistance, and code review

Assessment Alignment: The Technical Assessment requires students to transform a partially completed OpenGL program into a small interactive scene containing at least three visible objects, at least two primitive types, and at least three colors. One object

must be converted to a vertex array, one keyboard-controlled interaction must be implemented, and a user instruction must be rendered in the application window.

Syllabus Alignment: Modules 1–2 establish the OpenGL application structure, geometric primitives, vertices, lines, filled primitives, and colors; Module 3 develops GLUT text rendering and callback-based interaction; and Module 4 introduces vertex arrays and buffer-related rendering concepts. This progression supports the integrated requirements of Technical Assessment 1.

Coverage: Modules 1–4

Mode: Group Activity — 4 students

AI Classification: AI-Assisted

Development Time: 50 minutes

Code Defense: Approximately 5–6 minutes/group

Points: 50

Laboratory Activity:

Your group is provided with a **partially completed** [OpenGL starter program](#) that already:

- initializes GLUT;
- creates a window;
- contains a working `display()` function;
- draws one basic graphical object; and
- successfully compiles before modification.

Your task is to transform the starter program into a small **interactive OpenGL scene** demonstrating concepts from Modules 1–4.

The emphasis is **not on creating a complex artwork**. Your group will be assessed on whether you can correctly modify, integrate, and explain the required OpenGL concepts. Modules 1–2 establish the basic OpenGL application, primitives, vertices, lines, filled primitives, and colors, while Modules 3–4 introduce GLUT interaction and vertex-array rendering.

Programming Requirements

Within the allotted time, modify the starter program so that it satisfies **four requirements**.

Requirement 1 — Complete the Scene

Add at least **two additional visible objects** to the existing scene.

The completed scene must therefore contain at least **three visible objects**.

Across the scene, demonstrate at least **two different OpenGL primitive types**, such as:

GL_POINTS, GL_LINES, GL_LINE_LOOP, GL_TRIANGLES, GL_QUADS, or GL_POLYGON.

Use at least **three visibly different colors**.

Requirement 2 — Convert One Object to a Vertex Array

Choose **one object** and render it using a vertex array.

The implementation should demonstrate:

```
glEnableClientState(GL_VERTEX_ARRAY)
glVertexPointer()
glDrawArrays()
```

Students are **not required to use glDrawElements()** in this timed laboratory.

This keeps the requirement manageable while directly assessing Module 4.

Requirement 3 — Add One Interactive Control

Implement **one keyboard-controlled interaction** using:

```
glutKeyboardFunc()
```

For example:

- A / D → move an object left/right
- W / S → move an object up/down
- C → change an object's color
- R → reset its position
- another instructor-approved interaction

The callback should use:

```
glutPostRedisplay()
```

when the displayed scene needs to be updated.

Requirement 4 — Add a User Instruction

Display one short instruction in the window using the Module 3 text-rendering technique.

For example:

A/D - Move | R - Reset

Students may use:

```
glRasterPos2f()  
glutBitmapString()
```

Module 3 describes `glRasterPos2f()` as setting the position where bitmap text or pixel images are drawn and `glutBitmapString()` as rendering bitmap characters at that raster position.

AI-Assisted Rules

Generative AI **may be used only for defined tasks** during the 50-minute development period.

Permitted

Groups may ask AI to:

- explain an OpenGL/GLUT function;
- diagnose compiler errors;
- explain why graphical output is incorrect;
- suggest a **small code fragment** for one requirement;
- suggest vertex coordinates for a simple primitive;
- explain how to convert one immediate-mode object into a vertex array;
- review a callback written by the group.

For example:

"Explain why `glutPostRedisplay()` is needed after changing an object's position."

or:

"Here is our triangle code. Show us how the same three vertices could be represented as a vertex array."

Not Permitted

Students may **not** prompt:

"Complete Laboratory 1 for us."

"Generate the entire OpenGL program satisfying these requirements."

"Write the finished solution that we can submit."

AI assistance must remain **component-level rather than solution-level**.

AI Usage Log

Each group submits **one group AI log**:

AI Tool	What did we ask AI to help with?	What did our group change/verify?
ChatGPT/Codex/ etc.		
ChatGPT/Codex/ etc.		
ChatGPT/Codex/ etc.		

Maximum: three significant AI interactions.

If no AI was used:

Our group did not use generative AI during this laboratory activity.

No separate reflection is necessary. The **code defense replaces the reflection** and provides stronger evidence of learning.

Required Submission

Students must submit:

1. **C++ source file (.cpp)**
2. **Screenshot of the successfully running application**
3. **AI Usage Log**
4. **Short reflection (100–150 words)** addressing:
 - What part of the program was most challenging?
 - How did you solve the problem?
 - If AI was used, what did AI contribute?
 - What did you have to correct, modify, or verify yourself?

Rubric for Grading		
Criterion	Description	Points
Core Programming Requirements	Required graphical/programming features function correctly according to the laboratory specification.	20
Application of Module Concepts	Group correctly applies the relevant CS0045 concepts rather than merely producing the expected visual output.	10
Program Functionality & Code Quality	Application compiles/runs; modifications are organized and readable; no major unnecessary changes to starter code.	5
AI-Assisted Practice & Disclosure	AI is limited to permitted tasks; significant use is disclosed; group demonstrates evidence of evaluating rather than blindly accepting AI output.	5
Student Verification / Code Defense	Randomly selected members can explain implementation decisions and successfully respond to or perform instructor-requested modifications.	10
	TOTAL	50

Code Defense Scoring	
Performance	Score
Members accurately explain the code and live modification is completed independently	9–10
Good overall understanding; minor prompting required	7–8
Partial understanding; group knows what the program does but struggles to explain implementation	5–6
Significant dependence on memorized/AI-generated explanation; modification cannot be completed	2–4
Group cannot explain the submitted implementation	0–1

Important Grading Rule for AI-Assisted Work

The **quality of AI output itself is not graded**.

This assessment evaluates your ability to **use AI critically and transparently while demonstrating your own mastery** of OpenGL concepts from Modules 1–4, particularly geometric primitives, GLUT interaction, text rendering, and vertex arrays.

AI may assist with permitted component-level tasks, but your group must **understand, verify, and be able to explain any AI-assisted code** included in your solution.

A sophisticated AI-assisted program that the group cannot explain or modify should **not receive a higher score** than a simpler solution that correctly satisfies the requirements.

The **code defense verifies that the OpenGL knowledge and implementation decisions demonstrated in the submitted program belong to the students.**